Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Carrera de Ingeniería en Ciencias y Sistemas
Sistemas Operativos 2 — Ing. Otto Soto
Segundo semestre 2026

# REDXela

Simulador concurrente de un centro de distribución

## Manual técnico

**Brandon Gustavo Güinac Román**
Carné 201931217
Quetzaltenango, 27 de octubre de 2026

# Contenido

1. Introducción ..... 4
1.1 Alcance ..... 4
1.2 Requisitos de ejecución ..... 4
2. Arquitectura general ..... 6
2.1 Paquetes ..... 6
2.2 Hilos del sistema ..... 7
3. Modelo del problema ..... 8
3.1 Pedido ..... 8
3.2 Niveles de servicio y escala de tiempo ..... 8
3.3 Tipos de mercancía y recursos ..... 9
3.4 Recursos del centro ..... 9
3.5 Ciclo de vida de un pedido ..... 10
4. Recursos compartidos y exclusión mutua ..... 11
4.1 Dos niveles de semáforos ..... 11
4.2 ¿Por qué semáforos y no candados? ..... 11
4.3 Justicia (fairness) ..... 11
5. Planificación ..... 12
5.1 Políticas de atención ..... 12
5.2 Lista de espera ..... 12
5.3 Despachador ..... 13
5.4 Llegada de pedidos ..... 13
6. Prevención de interbloqueo ..... 14
6.1 Asignación todo o nada ..... 14
6.2 Por qué ordenar además de asignar todo o nada ..... 14
6.3 Inanición de Recepción ..... 15
6.4 Orden de los candados del propio programa ..... 15
7. Conflicto de recursos: detección y resolución ..... 16
7.1 El conflicto ..... 16
7.2 Modo incremental ..... 16
7.3 Garantizar el cruce con una barrera ..... 16
7.4 Detección: grafo de espera ..... 16
7.5 Presentación en la interfaz ..... 17
7.6 Resolución: expropiación con reversión ..... 17
8. Comunicación entre áreas (IPC) ..... 18
8.1 Propiedades garantizadas ..... 18
8.2 Flujo completo de una llegada de mercadería ..... 18
9. Administración del almacenamiento ..... 19
9.1 Capacidad ..... 19
9.2 Estrategia de asignación ..... 19
9.3 Retiro ..... 19
9.4 Concurrencia: lectores y escritores ..... 19
10. Otras estructuras concurrentes ..... 21
11. Comparación de políticas ..... 22
11.1 Análisis ..... 22
12. Manejo de excepciones ..... 23

13. Servidor web e interfaz ..... 24
13.1 Tiempo real con Server-Sent Events ..... 24
13.2 API REST ..... 24
14. Pruebas realizadas ..... 25
15. Limitaciones y mejoras posibles ..... 26
16. Referencias ..... 27

# 1. Introducción

Este manual describe la arquitectura interna de **REDXela**, un simulador del centro de distribución de comercio electrónico planteado en el proyecto final de Sistemas Operativos 2. Además de documentar la infraestructura, explica cada decisión de diseño y el fundamento teórico que la respalda: planificación, sincronización, exclusión mutua, semáforos, prevención y detección de interbloqueo, comunicación entre procesos y administración de almacenamiento.

El centro de distribución se modela como un pequeño sistema operativo. Los **pedidos** son los procesos; las estaciones de empaque, andenes, montacargas, encargados, estaciones de control de calidad y el escáner son los **recursos**; la lista de espera es la **cola de listos**; el despachador es el **planificador**; y el almacén es la **memoria** cuyo espacio hay que administrar.

## 1.1 Alcance

- Backend en Java 17 con hilos reales (`Thread`, `ThreadPoolExecutor`) y mecanismos reales de sincronización de `java.util.concurrent`: `Semaphore`, `ReentrantLock`, `Condition`, `ReentrantReadWriteLock`, `CountDownLatch`, variables atómicas y colecciones concurrentes.
- Interfaz web que muestra en tiempo real el estado de cada recurso, los pedidos en proceso, la lista de espera ordenada, el buffer entre áreas, el mapa del almacén, los conflictos y las estadísticas.
- Sin dependencias externas: el servidor HTTP es el que trae el JDK (`com.sun.net.httpserver`) y el JSON se serializa con una clase propia. Toda la concurrencia queda visible en el código fuente; ningún framework la oculta.

## 1.2 Requisitos de ejecución

| Elemento | Requisito |
|---|---|
| Java | JDK 17 o superior (probado con OpenJDK 17 y 21) |
| Compilación | Maven 3.8+ (`mvn package`) o los scripts `compilar.sh` / `compilar.bat`, que solo usan `javac` y `jar` |
| Navegador | Cualquier navegador moderno con soporte de Server-Sent Events (Chrome, Edge, Firefox) |
| Puerto | 8080 por defecto; se cambia con `java -jar redxela.jar 9090` o la variable `PORT` |
| Memoria | Menos de 200 MB de heap en uso normal |

### Compilar y ejecutar

```
# Con Maven
mvn package
java -jar target/redxela.jar

# Sin Maven (Linux/macOS)
./compilar.sh
java -jar redxela.jar
```

```
# Sin Maven (Windows)
compilar.bat
java -jar redxela.jar

# Luego abrir http://localhost:8080
```

# 2. Arquitectura general

El sistema tiene tres capas: la interfaz (navegador), el servidor web y el núcleo de simulación. El núcleo no conoce al servidor web; el servidor solo lee el estado del núcleo (a través de `VistaEstado`) e invoca operaciones públicas de la fachada `CentroDistribucion`.

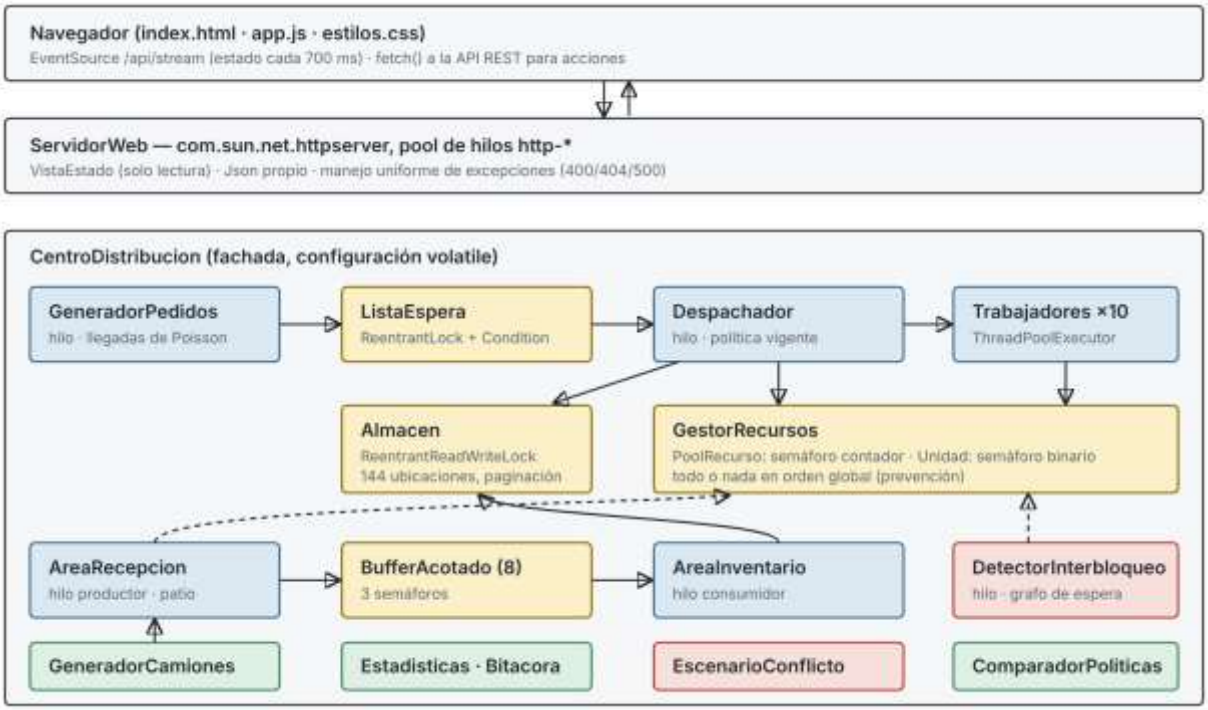



*Figura 1. Componentes principales y sus relaciones. Azul: hilos activos. Amarillo: estructuras compartidas protegidas. Rojo: interbloqueo. Verde: soporte.*

## 2.1 Paquetes

| Paquete | Contenido |
|---|---|
| `modelo` | Entidades: `Pedido`, `Cliente`, `Producto`, `LineaPedido`, `Camion`, y los enumerados `NivelServicio`, `TipoMercancia`, `TipoRecurso`, `EstadoPedido`. |
| `recursos` | `Unidad` (semáforo binario por unidad física), `PoolRecurso` (semáforo contador), `GestorRecursos` (asignación todo o nada) y `Operacion` (vértice del grafo de espera). |
| `planificacion` | `PoliticaPlanificacion` (cinco políticas), `ListaEspera` (cola con variable de condición), `Planificable` y `Reloj`. |
| `ipc` | `BufferAcotado` (productor-consumidor con tres semáforos) y `MensajeRecepcion`. |
| `areas` | `AreaRecepcion` (productor) y `AreaInventario` (consumidor). |
| `almacen` | `Almacen` (asignación paginada con candado lector-escritor), `Ubicacion`, `Lote`. |
| `interbloqueo` | `DetectorInterbloqueo` (grafo de espera) y `Conflicto`. |

| Paquete | Contenido |
|---|---|
| `nucleo` | `CentroDistribucion` (fachada), `Despachador`, `TrabajadorPedido`, generadores, `EscenarioConflicto`, `ComparadorPoliticas`, `Catalogo`. |
| `estadisticas` | Acumulados globales, por nivel y por política. |
| `web` | `ServidorWeb` (HTTP, API REST y SSE) y `VistaEstado` (fotografía del estado). |
| `util` | `Json`, `Bitacora`, `ValidacionException`. |

## 2.2 Hilos del sistema

Cada componente activo corre en su propio hilo. Todos se crean como hilos demonio (`setDaemon(true)`) para que la JVM pueda terminar aunque alguno quede bloqueado, y además existe un gancho de apagado (`Runtime.addShutdownHook`) que los detiene en orden al presionar Ctrl+C.

| Hilo | Cantidad | Función | Se bloquea en |
|---|---|---|---|
| `despachador` | 1 | Recorre la lista de espera según la política y asigna recursos | `Condition.await` de la lista de espera |
| `trabajador-N` | 10 | Procesa un pedido (simula el trabajo y libera los recursos) | `Thread.sleep`; en modo incremental, `Semaphore.acquire` de una unidad |
| `recepcion` | 1 | Descarga camiones del patio y envía avisos a Inventario | `LinkedBlockingDeque.pollFirst`, recursos, buffer lleno |
| `inventario` | 1 | Consume avisos del buffer y ubica la mercadería | `Semaphore.tryAcquire` del buffer |
| `detector-interbloqueo` | 1 | Construye el grafo de espera cada 500 ms | `Thread.sleep` |
| `generador-pedidos` | 1 | Crea pedidos con llegadas de Poisson | `Thread.sleep` (tiempo exponencial) |
| `reposicion` | 1 | Envía camiones cuando un producto baja de 15 unidades | `Thread.sleep` |
| `http-N` | variable | Atiende peticiones y conexiones SSE | E/S de red |
| `escenario-*` | 2 por escenario | Operaciones del conflicto provocado | Semáforos de las unidades en disputa |

> **¿Por qué 10 trabajadores?** Todo tipo de mercancía necesita al menos un encargado de bodega y hay exactamente 10. Por lo tanto nunca puede haber más de 10 pedidos en proceso al mismo tiempo; un pool más grande tendría hilos ociosos y uno más pequeño limitaría artificialmente el paralelismo.

# 3. Modelo del problema

## 3.1 Pedido

Cada pedido guarda la información mínima exigida y otros datos que la simulación necesita:

| Campo | Descripción |
|---|---|
| `id` | Identificador único `P-00001`, generado con un `AtomicInteger` (no se repite aunque dos hilos creen pedidos a la vez). |
| `cliente`, `nivel` | Cuenta que hizo el pedido y su nivel de servicio contratado; el nivel se hereda de la cuenta. |
| `lineas` | Productos y unidades de cada uno; cada producto tiene su tipo de mercancía. |
| `totalUnidades` | Suma de unidades: es la estimación base del trabajo, como pide el enunciado. |
| `tipos`, `recursosRequeridos` | Tipos presentes y la combinación de recursos resultante (máximo por recurso entre los tipos). |
| `duracionEstimadaMs` | 1 500 ms + Σ(unidades × 300 ms × factor del tipo). |
| `estado`, `motivoEspera` | Estado actual y la razón legible por la que espera. |
| `llegadaMs`, `inicioMs`, `finMs` | Marcas de tiempo para medir espera y procesamiento. |
| `progreso`, `reintentos`, `stockReservado` | Avance 0..1, veces que regresó por un conflicto y si su mercadería ya salió del almacén. |

Los datos que definen al pedido son `final` e inmutables. Los que cambian son `volatile`: los escribe un solo hilo a la vez (el despachador o el trabajador) y los leen otros (la interfaz), así que basta garantizar **visibilidad** entre hilos y no hace falta exclusión mutua.

## 3.2 Niveles de servicio y escala de tiempo

Para que los plazos de 10 a 120 minutos puedan observarse en una demostración se usa una escala de **1 minuto simulado = 1 segundo real**. Una hora simulada dura un minuto real. La clase `Reloj` mide los minutos simulados desde el arranque del programa para que los puntajes se lean como números pequeños.

| Nivel | Tipo de envío | Espera máxima | En tiempo real |
|---|---|---|---|
| 1 | Exprés | Casi instantáneo (se usa 1 min) | 1 s |
| 2 | Prioritario | 10 min | 10 s |
| 3 | Estándar | 30 min | 30 s |
| 4 | Programado | 60 min | 1 min |
| 5 | Económico | 120 min | 2 min |

## 3.3 Tipos de mercancía y recursos

El nivel de servicio decide **cuándo** se atiende un pedido; el tipo de mercancía decide **qué recursos** necesita. Se definieron seis tipos con combinaciones distintas entre sí:

| Tipo | Escáner | Andén | Montac. | C. calidad | Empaque | Encargados | Factor |
|---|---|---|---|---|---|---|---|
| General | | | | | 1 | 1 | 1.0 |
| Frágil | | | | 1 | 1 | 1 | 1.3 |
| Pesado | | 1 | 1 | | | 2 | 1.6 |
| Voluminoso | | 1 | 1 | | 1 | 1 | 1.4 |
| Perecedero | | 1 | | 1 | 1 | 1 | 1.1 |
| Alto valor | 1 | 1 | | 1 | 1 | 1 | 1.5 |

**Escenarios de contención requeridos.** Pesado y Voluminoso necesitan el mismo par de recursos compartidos (montacargas y andén); cuando llegan en momentos cercanos, el gestor resuelve la contención con la asignación todo o nada. Alto valor necesita el par (andén, escáner), el mismo que usa Recepción para descargar un camión, pero en orden inverso: es el par con el que se reproduce el interbloqueo.

Un pedido con varios tipos necesita el **máximo** de cada recurso entre sus tipos, no la suma: se procesa como un único trabajo y una estación de empaque basta para un producto frágil y uno general juntos.

## 3.4 Recursos del centro

| Recurso | Unidades | Orden global |
|---|---|---|
| Sistema de escaneo (compartido entre Recepción y despacho) | 1 | 1 |
| Área de carga (andén) | 2 | 2 |
| Montacargas | 3 | 3 |
| Estación de control de calidad | 4 | 4 |
| Estación de empaque | 6 | 5 |
| Encargado de bodega | 10 | 6 |
| Almacén | 144 ubicaciones × 30 unidades = 4 320 unidades | — |

## 3.5 Ciclo de vida de un pedido

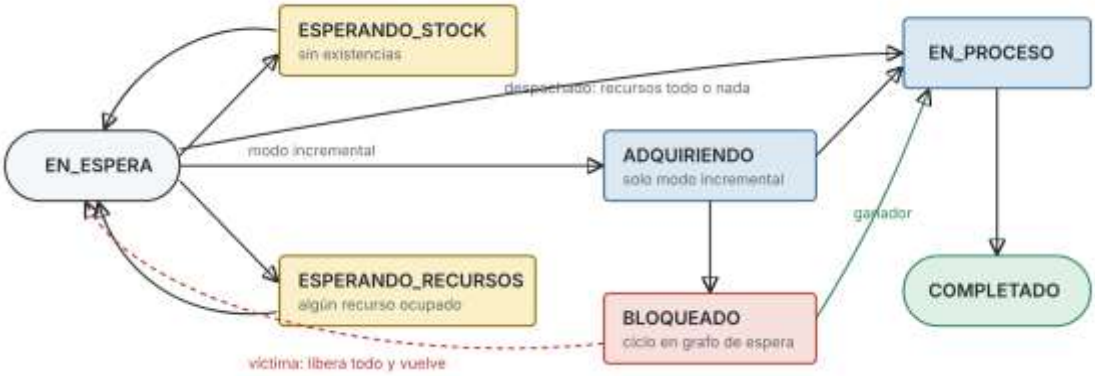



*Figura 2. Estados de un pedido. Equivalen a los estados de un proceso: listo, bloqueado, en ejecución y terminado.*

# 4. Recursos compartidos y exclusión mutua

## 4.1 Dos niveles de semáforos

Cada tipo de recurso es un `PoolRecurso` que combina:

- Un **semáforo contador** `Semaphore(N, true)` cuyo valor es exactamente el número de unidades libres. Es la definición de Dijkstra de un recurso con N instancias: `P()` toma una unidad, `V()` la devuelve.
- Un **semáforo binario por unidad** (`Unidad.mutex`, inicializado en 1) que garantiza exclusión mutua sobre cada unidad física e identifica cuál se usa: la interfaz puede mostrar "Andén 2 → P-00031".

La regla de consistencia es que toda unidad asignada consume un permiso del contador y al liberarla se devuelven ambos.

## 4.2 ¿Por qué semáforos y no candados?

El despachador toma los recursos de un pedido y es un hilo trabajador distinto el que los libera al terminar. Un `ReentrantLock` tiene dueño: solo el hilo que lo bloqueó puede desbloquearlo, y otro hilo obtendría `IllegalMonitorStateException`. Un semáforo no tiene dueño, así que permite esa liberación cruzada de forma legítima. El control de "quién es dueño" lo hace la aplicación en `Unidad.liberar()`, que verifica que quien libera sea la operación dueña.

## 4.3 Justicia (fairness)

Todos los semáforos se crean con `fair = true`: los hilos que esperan reciben el permiso en orden FIFO, lo que evita la inanición en la espera. Hay un detalle importante: `tryAcquire()` sin tiempo **ignora** la justicia (puede "colarse" delante de quienes esperan). Por eso `Unidad.intentarTomar()` revisa primero `hasQueuedThreads()`: si alguien espera esa unidad, se respeta su turno. Esto es lo que garantiza que, al resolver un interbloqueo, la operación ganadora reciba el recurso que liberó la víctima y no otra que llegue después.

```
boolean intentarTomar() {
    return !mutex.hasQueuedThreads() && mutex.tryAcquire();
}
```

# 5. Planificación

## 5.1 Políticas de atención

Cada política asigna un **puntaje** a cada pedido en el instante actual; menor puntaje se atiende antes. El puntaje se muestra en la lista de espera para que se entienda el orden. Se implementan como un `enum` con un método abstracto, de modo que la simulación en vivo y el comparador usan exactamente el mismo código.

| Política | Puntaje | Equivalente en CPU | Propiedades |
|---|---|---|---|
| FIFO | minuto de llegada | FCFS | Justa en orden, ignora la urgencia del cliente. |
| Prioridad estricta | nivel × $10^6$ + llegada | Prioridad no expropiativa | Los niveles 1–2 casi no esperan; los 4–5 pueden sufrir inanición. |
| Envejecimiento | nivel − espera / 15 | Prioridad con aging | Cada 15 min de espera mejoran un nivel. Ningún pedido espera para siempre. |
| Menos artículos | unidades × 10 + nivel | SJF | Minimiza la espera promedio de pedidos pequeños; los grandes pueden esperar mucho. |
| Plazo más próximo | llegada + espera máxima del nivel | EDF (tiempo real) | Maximiza los pedidos atendidos dentro de su plazo mientras el sistema no esté saturado. |

## 5.2 Lista de espera

`ListaEspera` no usa una `PriorityQueue` porque el orden depende del instante actual (en envejecimiento la prioridad cambia sola mientras se espera) y de la política, que puede cambiarse en caliente. En cada vuelta el despachador pide una copia ordenada.

La lista está protegida por un `ReentrantLock` justo con una variable de condición `huboCambios`. El despachador duerme en `await` (sin consumir CPU, sin espera activa) hasta que alguien llama a `notificar()`: llega un pedido, se libera un recurso o entra mercadería. Una bandera `pendiente` evita la **pérdida de avisos** (lost wake-up): si el aviso llega mientras el despachador estaba revisando la lista y no esperando, la siguiente espera retorna de inmediato. Además, la espera tiene un límite de 400 ms para reevaluar puntajes que cambian con el tiempo.

```
public void esperarCambios(long maxMs) throws InterruptedException {
    cerrojo.lock();
    try {
        if (!pendiente) huboCambios.await(maxMs, TimeUnit.MILLISECONDS);
        pendiente = false;
    } finally {
        cerrojo.unlock();
    }
}
```

## 5.3 Despachador

En cada vuelta el despachador recorre la lista ordenada y, para cada pedido:

1. Si no tiene su mercadería reservada, consulta existencias. Si faltan, lo marca `ESPERANDO_STOCK` con los productos faltantes y sigue con el siguiente.
2. Intenta asignarle todos sus recursos de una sola vez (todo o nada). Si falta alguno, lo marca `ESPERANDO_RECURSOS` con el detalle ("Falta: Andén 0/2 libres") y sigue.
3. Retira su mercadería del almacén de forma atómica. Si entre la consulta y el retiro otro hilo la tomó, devuelve los recursos.
4. Lo saca de la lista, registra la operación y lo entrega al pool de trabajadores.

**Decisión de diseño: relleno (backfilling).** Si el primer pedido de la lista no puede avanzar por falta de recursos, se intenta con el siguiente en lugar de detener todo. Un pedido General no tiene por qué esperar a que se libere un andén que solo necesita un Pesado. Esto aumenta el uso de los recursos, pero puede retrasar a pedidos grandes que nunca encuentran todos sus recursos libres a la vez; la política con envejecimiento lo compensa elevando su prioridad con el tiempo, y la reserva de andén/escáner para Recepción (sección 6.3) evita el caso más crítico.

## 5.4 Llegada de pedidos

El generador automático modela las llegadas como un **proceso de Poisson**: el tiempo entre pedidos se sortea con distribución exponencial de media 60 s / tasa, donde la tasa está en pedidos por hora simulada. Es el modelo estándar de la teoría de colas para llegadas aleatorias e independientes, y produce ráfagas y huecos realistas. El 10 % de los pedidos proviene de un cliente nuevo que primero crea su cuenta y elige su nivel.

```
long espera = Math.round(-Math.log(1 - azar.nextDouble()) * 60_000.0 / tasa);
```

# 6. Prevención de interbloqueo

Un interbloqueo requiere que se cumplan a la vez las cuatro condiciones de Coffman. En modo seguro (el modo por defecto) el sistema rompe dos de ellas:

| Condición | ¿Se rompe? | Cómo |
|---|---|---|
| Exclusión mutua | No | Es inherente: un andén no puede cargar dos camiones a la vez. |
| Retener y esperar | Sí | Asignación todo o nada: si falta un recurso se devuelven los ya tomados y la operación no retiene nada mientras espera. |
| Sin expropiación | No (en modo seguro) | Nadie quita recursos a una operación en curso. Sí se usa expropiación para resolver un interbloqueo detectado (sección 7). |
| Espera circular | Sí | Orden global de adquisición: escáner → andén → montacargas → control de calidad → empaque → encargado. El `EnumMap` itera siempre en ese orden. |

## 6.1 Asignación todo o nada

```
public boolean intentarAsignar(Operacion op, Map<TipoRecurso,Integer> requeridos, boolean
esRecepcion) {
    Map<TipoRecurso,Integer> ordenados = new EnumMap<>(requeridos); // orden global
    cerrojo.lock();
    try {
        List<Unidad> tomadas = new ArrayList<>();
        for (var e : ordenados.entrySet()) {
            List<Unidad> parte = pools.get(e.getKey()).intentarTomar(e.getValue(), op);
            if (parte == null) {                    // falta este recurso
                for (Unidad u : tomadas) pools.get(u.tipo()).liberar(u, op);
                return false;                       // no retiene nada
            }
            tomadas.addAll(parte);
        }
        return true;
    } finally { cerrojo.unlock(); }
}
```

Toda la asignación ocurre dentro de la sección crítica de un `ReentrantLock` del gestor, así dos asignaciones múltiples nunca se intercalan. Las esperas bloqueantes (Recepción) usan una variable de condición `seLiberoAlgo` del mismo candado: se reintenta cada vez que alguien libera un recurso, sin espera activa.

## 6.2 Por qué ordenar además de asignar todo o nada

Con todo o nada ya no hay retención y espera, lo que por sí solo impide el interbloqueo. El orden global se mantiene como defensa en profundidad: si en el futuro alguna operación pidiera recursos de forma incremental pero respetando el orden, tampoco podría formarse un ciclo. Los recursos más escasos

(escáner y andenes) van primero porque son los que con más probabilidad fallan, y así el intento se aborta pronto.

## 6.3 Inanición de Recepción

Recepción necesita andén y escáner, igual que muchos pedidos. Con relleno, el despachador podría tomar siempre el andén apenas se libera y Recepción nunca encontraría los cuatro recursos libres a la vez. Para evitarlo, mientras haya un camión esperando en modo seguro (`recepcionesEsperando > 0`) los pedidos no pueden tomar andén ni escáner. Se le da prioridad a la entrada de mercadería porque, además, sin mercadería los pedidos tampoco avanzan.

## 6.4 Orden de los candados del propio programa

El propio código también podría interbloquearse si dos hilos tomaran dos candados en orden inverso. Se evitó así:

- El despachador no tiene tomado el candado de la lista mientras pide recursos: trabaja sobre una copia ordenada. Es el único que saca pedidos de la lista, así que la copia es segura.
- Al liberar recursos, el gestor avisa a los observadores (el despachador) **después** de soltar su propio candado, nunca anidado.
- Inventario suelta el candado de escritura del almacén antes de notificar a la lista de espera.

En consecuencia ningún hilo tiene tomados dos de estos candados al mismo tiempo, y no puede haber espera circular entre ellos.

# 7. Conflicto de recursos: detección y resolución

## 7.1 El conflicto

Dos operaciones necesitan el mismo par de recursos en orden inverso:

```
Recepción (camión):      toma ESCÁNER  → pide ANDÉN k
Despacho de alto valor:  toma ANDÉN k  → pide ESCÁNER
```

En el mundo real ambos órdenes tienen sentido: el camión se escanea al bajarse la mercadería y luego se lleva a un andén; el pedido de alto valor se coloca primero en el andén y se escanea al salir para el seguro de envío. Si cada una toma su primer recurso, ambas quedan esperando para siempre.

## 7.2 Modo incremental

Para que el conflicto pueda ocurrir, estas operaciones piden los dos recursos uno por uno con `PoolRecurso.tomarEspecifica()`, que se bloquea con `Semaphore.acquire()` reteniendo lo que ya tienen. Esto sucede en dos situaciones:

- **Escenario provocado** (botón "Provocar conflicto"): crea un pedido de alto valor y un camión asignados al mismo andén y los ejecuta en hilos propios en modo incremental, aunque la prevención global esté activa. El resto del centro sigue funcionando.
- **Prevención desactivada**: Recepción y todos los despachos de alto valor usan el modo incremental. Bajo carga, el interbloqueo aparece solo.

## 7.3 Garantizar el cruce con una barrera

Sin coordinación, el interbloqueo dependería de la suerte en la planificación de hilos de la JVM. El escenario usa un `CountDownLatch(2)` como **barrera**: cada operación, después de tomar su primer recurso, hace `countDown()` y espera en `await()` a que la otra también lo haya tomado. Al liberarse la barrera ambas piden el segundo recurso y el ciclo queda formado con certeza.

## 7.4 Detección: grafo de espera

El hilo `detector-interbloqueo` construye cada 500 ms el **grafo de espera** (wait-for graph): los vértices son las operaciones y hay una arista A → B cuando A espera una unidad que retiene B. Como toda espera bloqueante es por una unidad concreta (una instancia única), un **ciclo** en este grafo es condición necesaria y suficiente de interbloqueo (Silberschatz, capítulo 7).

Cada operación espera a lo sumo una unidad, así que cada vértice tiene a lo sumo una arista de salida; basta seguir la cadena desde cada vértice hasta que se corta (sin ciclo) o se repite un vértice (ciclo). Para no reportar una "foto" inconsistente tomada justo mientras el grafo cambiaba (se lee sin detener a los demás hilos), un ciclo solo se reporta si aparece en dos pasadas seguidas.

```
for (Operacion inicio : gestor.operaciones()) {
    List<Operacion> camino = new ArrayList<>();
    Operacion actual = inicio;
    while (actual != null) {
        int idx = camino.indexOf(actual);
        if (idx >= 0) { /* ciclo = camino[idx..fin] */ break; }
        camino.add(actual);
```

```
        Unidad espera = actual.esperandoPor();
        actual = (espera == null) ? null : espera.duenio(); // arista actual → dueño
    }
}
```

## 7.5 Presentación en la interfaz

Al detectarse, la interfaz muestra una alerta con franjas de peligro que indica qué recursos están involucrados, qué operaciones están bloqueadas, qué retiene y qué espera cada una, y dibuja el grafo de asignación de recursos del ciclo.




*Figura 3. Interbloqueo detectado: el camión retiene el escáner y espera el andén; el pedido retiene el andén y espera el escáner.*

## 7.6 Resolución: expropiación con reversión

El usuario elige qué operación continúa. A las demás:

1. Se les marca como víctimas y se interrumpe su hilo (`Thread.interrupt()`).
2. El hilo sale de `Semaphore.acquire()` con `InterruptedException`.
3. El bloque `finally` libera **todas** sus unidades.
4. El pedido regresa a la lista de espera conservando su hora de llegada original (no pierde antigüedad) y su mercadería ya reservada. El camión regresa al frente del patio. Nada se pierde.

Como la ganadora está formada en la cola justa del semáforo de la unidad liberada, la recibe inmediatamente y continúa. Este mecanismo rompe la condición de "sin expropiación" solo en el momento de la recuperación.

# 8. Comunicación entre áreas (IPC)

Recepción y Control de Inventario son dos áreas independientes que no modifican las estructuras internas de la otra. Recepción solo deposita mensajes (`MensajeRecepcion`: camión, producto y cantidad) en un buffer compartido; Inventario los extrae y procesa a su propio ritmo. Es el problema clásico del **productor-consumidor con buffer acotado**, resuelto con tres semáforos.




*Figura 4. Buffer circular de 8 ranuras entre Recepción e Inventario.*

```
productor:  P(espaciosLibres); P(mutex); poner; V(mutex); V(elementos)
consumidor: P(elementos);      P(mutex); sacar; V(mutex); V(espaciosLibres)
```

## 8.1 Propiedades garantizadas

- **Sin pérdida de datos.** Si el buffer está lleno, el productor espera; nunca se sobrescribe un mensaje no leído. El mensaje sigue en manos de Recepción hasta que se deposita.
- **Sin bloqueo indefinido.** `ofrecer()` espera como máximo 1 s y `tomar()` 0.5 s. Si vence el tiempo, el hilo recupera el control, registra la situación en la bitácora ("buffer lleno"), revisa si debe apagarse y reintenta.
- **Orden correcto de los P().** `P(espaciosLibres)` siempre antes de `P(mutex)`. Al revés, un productor podría dormirse con el mutex tomado y bloquear también al consumidor: un interbloqueo clásico.
- **Recepción no acapara recursos mientras espera.** Libera el escáner, el andén y su cuadrilla antes de intentar depositar el aviso.
- **Mensajes inmutables.** `MensajeRecepcion` es un `record`; pasar la referencia entre hilos es seguro y el semáforo establece la relación happens-before.

## 8.2 Flujo completo de una llegada de mercadería

1. El camión llega al patio (`LinkedBlockingDeque` sin límite: un camión nunca se rechaza).
2. Recepción obtiene escáner, andén, montacargas y encargado; descarga y escanea.
3. Libera los recursos y deposita el aviso en el buffer.
4. Inventario toma el aviso, simula su registro (tiempo configurable) y crea un lote en el almacén asignando sus ubicaciones.
5. Si no cupo todo, el resto queda "pendiente de ubicar" y se reintenta cuando salen pedidos.
6. Inventario notifica a la lista de espera: el despachador reevalúa los pedidos que esperaban ese producto.

# 9. Administración del almacenamiento

Un lote puede ocupar varias ubicaciones y estas no tienen que ser contiguas. El problema es el mismo que la administración de memoria de un sistema operativo, y se resolvió con **paginación**:

| Memoria | Almacén REDXela |
|---|---|
| Marco de página (tamaño fijo) | Ubicación del rack: pasillo-nivel-posición, capacidad 30 unidades |
| Proceso | Lote recibido en un camión |
| Página | Bloque de hasta 30 unidades del lote |
| Tabla de páginas | Tabla de ubicaciones del lote (lista ordenada de ubicaciones) |
| Lista de marcos libres | `TreeSet<Ubicacion>` ordenado por dirección |
| Fragmentación interna | Espacio sin usar en la última ubicación de cada lote |

## 9.1 Capacidad

El almacén tiene 6 pasillos (A–F) × 4 niveles × 6 posiciones = 144 ubicaciones de 30 unidades cada una: 4 320 unidades en total. Cada ubicación tiene una dirección lineal (0..143) y un código legible como `B-3-05`.

## 9.2 Estrategia de asignación

Al guardar un lote se divide en bloques de 30 unidades y cada bloque va a la ubicación libre de **menor dirección** (primer ajuste sobre la lista de libres). Al principio los lotes quedan contiguos; a medida que salen pedidos y se liberan ubicaciones en distintos puntos, los lotes nuevos quedan repartidos en pasillos y niveles distintos. La bitácora y el mapa indican cuándo un lote es "no contiguo".

- **Ventaja:** no existe fragmentación externa; cualquier ubicación libre sirve, y un lote grande cabe mientras la suma de ubicaciones libres alcance.
- **Costo:** fragmentación interna en la última ubicación de cada lote (la interfaz muestra el total) y la necesidad de mantener una tabla de ubicaciones por lote.

## 9.3 Retiro

El retiro de las mercaderías de un pedido es **atómico**: o hay existencias para todas sus líneas y se retiran todas, o no se retira nada. Así nunca queda un pedido a medias acaparando existencias que otro podría usar. Se despacha FIFO por lotes (lo más antiguo primero, importante para perecederos) y dentro del lote se vacía primero la última ubicación, normalmente la parcialmente llena, lo que reduce la fragmentación interna. Las ubicaciones que quedan vacías regresan a la lista de libres.

## 9.4 Concurrencia: lectores y escritores

El almacén lo escriben Inventario (al guardar) y el despachador (al retirar), y lo leen el despachador (consultas de stock) y la interfaz (el mapa). Se protege con un `ReentrantReadWriteLock` justo: muchas lecturas pueden ocurrir en paralelo y una escritura es exclusiva. La interfaz lee todo el almacén dentro de una sola sección de lectura (`Almacen.leer(...)`) para no dibujar estados intermedios.




*Figura 5. Mapa del almacén. La parte clara de cada ubicación es espacio libre (fragmentación interna). Abajo, la tabla de ubicaciones del lote seleccionado.*

# 10. Otras estructuras concurrentes

| Estructura | Dónde | Por qué |
|---|---|---|
| `volatile` | Estado del pedido, configuración, dueño de una unidad | Un solo escritor a la vez; solo se necesita visibilidad inmediata entre hilos. |
| `AtomicInteger` / `AtomicLong` | Ids, contadores del buffer y de estadísticas | Incremento atómico sin candado (instrucciones CAS). |
| `synchronized` | `Unidad.asignar/liberar`, `Estadisticas` | Varios campos relacionados deben cambiar juntos y verse consistentes. |
| `ConcurrentHashMap` | Registro de operaciones, clientes | Escrituras desde varios hilos; lecturas sin bloquear. |
| `CopyOnWriteArrayList` | Unidades retenidas, conflictos, lista de clientes | Muchas lecturas y pocas escrituras; se recorre sin `ConcurrentModificationException`. |
| `ConcurrentLinkedDeque` | Bitácora, últimos completados | Cola no bloqueante: registrar un evento nunca detiene a un hilo de trabajo. |
| `LinkedBlockingDeque` | Patio de camiones | Cola bloqueante doble: permite devolver al frente un camión revertido. |
| `CountDownLatch` | Escenario de conflicto | Barrera que garantiza el cruce de las dos operaciones. |

# 11. Comparación de políticas

Comparar en la simulación en vivo no es justo, porque cada política ve pedidos distintos en momentos distintos. Por eso existe `ComparadorPoliticas`: genera **una sola** secuencia de pedidos con semilla fija (mismos tipos, cantidades y llegadas, usando las mismas reglas del generador) y la procesa con cada política mediante una **simulación de eventos discretos**. El reloj salta de evento en evento (llegada o fin de un pedido) sin dormir, de modo que cientos de pedidos se simulan en milisegundos. El despacho usa las mismas reglas que el despachador real: orden por política, asignación todo o nada y relleno.

Resultado con 600 pedidos a 50 pedidos por hora simulada (semilla 2026), una carga que lleva los andenes a cerca del 85 % de ocupación:

| Política | Espera prom. | Espera máx. | En plazo | N1 | N2 | N3 | N4 | N5 |
|---|---|---|---|---|---|---|---|---|
| FIFO | 5.7 | 31.2 | 87.3 % | 5.8 | 6.4 | 5.5 | 5.0 | 6.1 |
| Prioridad estricta | 8.2 | 290.4 | 92.8 % | 1.2 | 1.4 | 3.1 | 7.8 | 33.1 |
| Envejecimiento | 5.9 | 60.4 | 95.3 % | 0.9 | 1.9 | 4.7 | 7.6 | 15.0 |
| Menos artículos | 7.9 | 562.2 | 90.2 % | 4.1 | 5.0 | 8.5 | 5.6 | 15.6 |
| Plazo más próximo | 6.8 | 109.8 | 95.5 % | 1.0 | 1.5 | 3.7 | 8.0 | 22.7 |

Tiempos en minutos simulados. N1..N5 = espera promedio por nivel de servicio.

## 11.1 Análisis

- **FIFO** tiene la menor espera máxima, pero trata igual a un cliente Exprés que a uno Económico: el nivel 1 espera casi 6 minutos y solo el 87 % sale en plazo.
- **Prioridad estricta** atiende muy rápido a los niveles 1–2, pero un pedido Económico llegó a esperar 290 minutos: es la inanición que predice la teoría.
- **Menos artículos (SJF)**, con relleno, deja atrás a los pedidos grandes: su espera máxima es la peor (562 min). Con carga moderada sí logra la menor espera promedio.
- **Plazo más próximo (EDF)** maximiza el cumplimiento de plazos, aunque su espera máxima crece cuando la carga se acerca a la saturación.
- **Envejecimiento** casi empata con EDF en cumplimiento (95.3 %), conserva tiempos bajos para Exprés y Prioritario, y su espera máxima es la segunda mejor.

> **Recomendación para producción:** Prioridad con envejecimiento (la política por defecto). Respeta el nivel contratado, cumple los plazos casi tan bien como EDF y garantiza que ningún pedido sufra inanición. Con carga muy alta conviene revisar EDF; con carga baja todas se comportan de forma parecida porque casi no se forma cola.

# 12. Manejo de excepciones

| Situación | Tratamiento |
|---|---|
| Datos inválidos desde la interfaz | `ValidacionException` con mensaje legible; el servidor responde 400 (o 404 si el id no existe). Se valida cantidad, nivel, correo, producto, cliente, rangos de configuración. |
| JSON mal formado | `Json.JsonException` → 400 con la posición del error. |
| Error inesperado en una petición | Se registra en bitácora y consola y se responde 500, sin tumbar el servidor. |
| Error en un hilo de la simulación | Cada bucle (`despachador`, `recepcion`, `inventario`, `detector`, generadores) captura `Exception`, la registra y continúa: un pedido problemático no detiene al centro. |
| Interrupción de un hilo | Si la operación es víctima de un conflicto se revierte; si no, se restablece la marca de interrupción y el hilo termina (apagado). |
| Recursos al fallar un pedido | `TrabajadorPedido` libera todo en un bloque `finally`, ocurra lo que ocurra. |
| Excepción no capturada | Manejador global `Thread.setDefaultUncaughtExceptionHandler`: ningún hilo muere sin dejar rastro. |
| Puerto ocupado | Mensaje claro al iniciar con la instrucción para usar otro puerto. |
| Conexión SSE cerrada | La `IOException` se trata como fin normal de la conexión y se libera el hilo. |

# 13. Servidor web e interfaz

## 13.1 Tiempo real con Server-Sent Events

La interfaz abre una conexión `EventSource` a `/api/stream`. El servidor deja la respuesta abierta (transferencia por bloques) y escribe el estado completo en JSON cada 700 ms. Es más simple que WebSocket porque la comunicación en tiempo real solo va del servidor al navegador; las acciones del usuario viajan por peticiones REST normales. Si la conexión se cae, el navegador reintenta solo.

El estado se calcula una sola vez por intervalo aunque haya varias pestañas abiertas (caché de 300 ms protegida por un monitor) y se invalida después de cada acción para que el efecto se vea de inmediato. La construcción del estado (`VistaEstado`) solo lee: campos `volatile`, colecciones concurrentes y el almacén bajo su candado de lectura.

El servidor usa un pool `newCachedThreadPool`: cada conexión SSE ocupa un hilo mientras la pestaña está abierta, y un pool que crece bajo demanda evita que esas conexiones bloqueen la API.

## 13.2 API REST

| Método y ruta | Uso |
|---|---|
| GET `/api/catalogo` | Productos, tipos, niveles, políticas, recursos y clientes |
| GET `/api/estado` | Estado completo (lo mismo que envía el stream) |
| GET `/api/stream` | Server-Sent Events con el estado cada 700 ms |
| POST `/api/clientes` | Crear cuenta: `{nombre, correo, nivel}` |
| POST `/api/pedidos` | Registrar pedido: `{clienteId \| nuevoCliente, lineas:[{productoId, cantidad}]}` |
| POST `/api/mercaderia` | Llegada de camión: `{productoId, cantidad, anden}` |
| POST `/api/mercaderia/rafaga` | Varios camiones a la vez: `{camiones}` |
| POST `/api/config` | `{politica, prevencion, generador, tasa, reposicion, factorTiempo, retardoInventario}` |
| POST `/api/conflictos/provocar` | Escenario de interbloqueo: `{anden, productoId}` |
| POST `/api/conflictos/resolver` | `{conflictoId, ganadorId}` |
| POST `/api/estadisticas/reiniciar` | Pone en cero estadísticas y ocupación |
| POST `/api/comparar` | Comparador de políticas: `{pedidos, tasa, semilla}` |

# 14. Pruebas realizadas

| Prueba | Resultado |
|---|---|
| Carga sostenida en modo seguro (120 pedidos/h, velocidad 4×, cambiando de política cada 4 s y con ráfagas de camiones) | 113 pedidos completados, 60 camiones, 0 interbloqueos, 0 errores; el número de unidades libres coincide siempre con el valor del semáforo contador. |
| Escenario provocado, gana el pedido | Interbloqueo detectado en ~1 s; el camión se revierte, vuelve al patio y luego se descarga normalmente. |
| Escenario provocado, gana el camión | El pedido libera todo, vuelve a la lista con su antigüedad y se completa después. |
| Prevención desactivada con carga alta | El interbloqueo aparece solo entre Recepción y un pedido de alto valor; se resuelve desde la interfaz. |
| Inventario lento (6 s por aviso) y ráfaga de 18 camiones | El buffer llega a 8/8; Recepción espera (contador "esperas por lleno"), ningún aviso se pierde y el patio se vacía. |
| Datos inválidos (cliente inexistente, cantidad 0, JSON roto) | Respuestas 400/404 con mensaje claro; el sistema sigue operando. |

# 15. Limitaciones y mejoras posibles

- El estado vive en memoria: al cerrar el programa se pierde. Una base de datos permitiría auditar pedidos históricos.
- El detector solo modela esperas por unidades concretas. Las esperas de asignación múltiple (todo o nada) no retienen recursos, por lo que no pueden formar parte de un ciclo y no necesitan modelarse.
- La política con envejecimiento usa un factor fijo de 15 minutos por nivel; podría ajustarse automáticamente según la carga.
- El comparador asume existencias suficientes; podría incluir reposición para estudiar pedidos que esperan stock.

# 16. Referencias


- Silberschatz, A., Galvin, P. y Gagne, G. Operating System Concepts, 10.ª ed. Wiley. Capítulos 5 (planificación), 6–7 (sincronización e interbloqueo) y 9 (memoria principal).
- Tanenbaum, A. y Bos, H. Modern Operating Systems, 4.ª ed. Pearson. Capítulos 2 y 6.
- Goetz, B. et al. Java Concurrency in Practice. Addison-Wesley.
- Documentación de Java 17: paquete `java.util.concurrent` y módulo `jdk.httpserver`.